

Task 1: Hardware-Conscious 2D Convolution Optimization

1 Introduction

In this report, we analyze the performance of five optimization techniques applied to a single-channel 2D convolution kernel: Loop Reordering, Loop Unrolling, Cache Tiling, SIMD (AVX2), and a combined Unrolling + SIMD optimized kernel. All variants are compiled with identical flags (`-O2 -fno-tree-vectorize -mavx2 -mfma`), so no auto-vectorization occurs and every SIMD instruction is hand-written. Performance is measured as speedup over the naive reference, along with L1D cache misses and MPKI (misses per thousand instructions).

2 Task 1A: Loop Reordering

Loop reordering changes the iteration order from the natural $oy \rightarrow ox \rightarrow ky \rightarrow kx$ to $ky \rightarrow kx \rightarrow oy \rightarrow ox$, hoisting the kernel weight lookup out of the inner loop.

```
1 memset(out, 0, H * W * sizeof(float));
2 for (int ky = 0; ky < K; ++ky)
3     for (int kx = 0; kx < K; ++kx) {
4         float kv = ker[ky * K + kx];           // loaded once per (ky,kx)
5         for (int oy = 0; oy < H; ++oy)
6             for (int ox = 0; ox < W; ++ox)
7                 out[oy*W+ox] += in[(oy+ky)*in_stride+(ox+kx)] * kv;
8     }
```

2.1 Analysis

The motivation behind loop reordering was to reduce cache misses in the inner loop. Both orderings have roughly the same number of memory accesses, but the reordered loop should achieve more cache hits due to better data reuse, as the inner loop now updates independent output elements sequentially. However, for large matrices (e.g., 2048×2048 , 16 MB), the repeated full-array passes cause *cache thrashing*. The output array is repeatedly evicted and refetched from DRAM on each of the K^2 passes, leading to performance degradation. This is reflected in the MPKI rising to 1.72 at 2048^2 — the highest of all variants — and the speedup falling to $0.84\times$ (slower than naive).

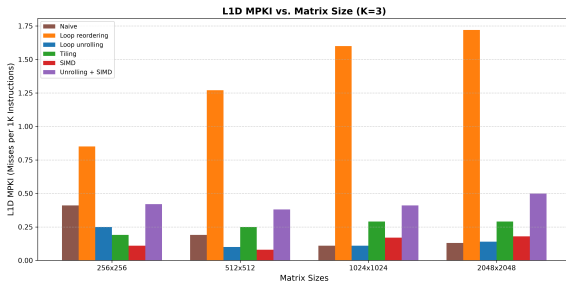


Figure 1: L1D MPKI vs. Matrix Size ($K = 3$).

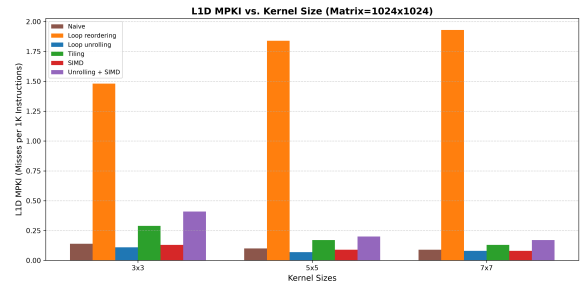


Figure 2: L1D MPKI vs. Kernel Size (1024×1024).

3 Task 1B: Loop Unrolling

The output column loop is unrolled by a factor of 4, computing four adjacent output pixels simultaneously with four independent accumulators.

```

1 for (int ox = 0; ox <= W - 4; ox += 4) {
2     float a0 = 0, a1 = 0, a2 = 0, a3 = 0;
3     for (int ky = 0; ky < K; ++ky)
4         for (int kx = 0; kx < K; ++kx) {
5             float wt = ker[ky * K + kx];
6             a0 += in[(oy+ky)*in_stride + (ox+kx)] * wt;
7             a1 += in[(oy+ky)*in_stride + (ox+1+kx)] * wt;
8             a2 += in[(oy+ky)*in_stride + (ox+2+kx)] * wt;
9             a3 += in[(oy+ky)*in_stride + (ox+3+kx)] * wt;
10        }
11    out[oy*W+ox]=a0; out[oy*W+ox+1]=a1;
12    out[oy*W+ox+2]=a2; out[oy*W+ox+3]=a3;
13 }

```

3.1 Analysis

The primary benefit of loop unrolling here is reducing branch instructions. By computing four adjacent output pixels in a single loop iteration, we significantly decrease the number of loop-control branches. This reduction minimizes pipeline stalls caused by branch prediction overhead or mispredictions, contributing to the observed speedup. The speedup is a consistent **2.1–2.4×** across *all* matrix and kernel sizes because the benefit is purely from instruction-level improvements rather than memory access changes.

4 Task 1C: Cache Tiling

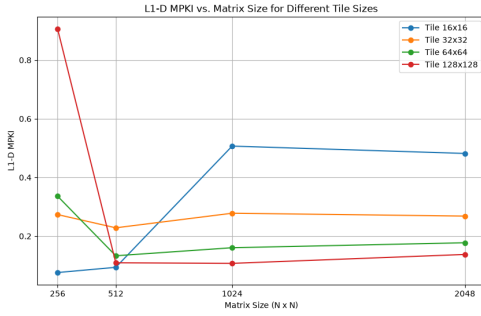


Figure 3: L1D MPKI vs. Matrix Size for different tile sizes.

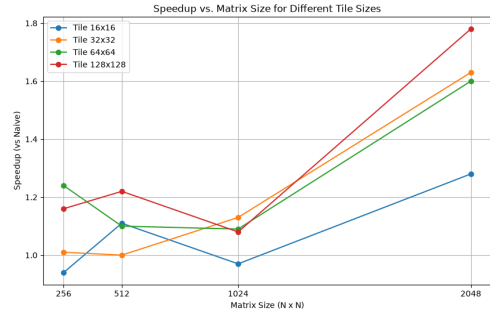


Figure 4: Speedup vs. Matrix Size for different tile sizes.

The output matrix is partitioned into 32×32 tiles processed one at a time, keeping the working set within the L1 data cache.

```

1 const int TILE = 32;
2 for (int ty = 0; ty < H; ty += TILE)
3     for (int tx = 0; tx < W; tx += TILE)
4         for (int oy = ty; oy < min(ty+TILE, H); ++oy)
5             for (int ox = tx; ox < min(tx+TILE, W); ++ox) {
6                 float acc = 0.f;
7                 for (int ky = 0; ky < K; ++ky)
8                     for (int kx = 0; kx < K; ++kx)
9                         acc += in[(oy+ky)*in_stride+(ox+kx)] * ker[ky*K+kx];
10                out[oy*W+ox] = acc;
11            }

```

4.1 Analysis

A 32×32 float tile is 4KB, which fits comfortably in L1 cache. The intent is to amortize kernel passes over a cache-resident input region. However, our tiling implementation uses the naive scalar inner loop, providing *no ILP or vectorization benefit*. The tile boundary arithmetic adds overhead and disrupts the sequential access pattern that the hardware prefetcher exploits in the naive kernel. As a result, tiling alone hurts performance here, consistently running *slower than naive* (0.66–0.86×). Tiling must be paired with SIMD or aggressive unrolling to deliver a payoff.

Deliverable Answers:

1. **Changes in L1-D MPKI (Naive to Tiled):** The L1-D MPKI slightly increased or remained comparable for most tile sizes compared to the naive version (which had an MPKI of ~ 0.11 - 0.19). For instance, tiling often yielded MPKIs around 0.13 to 0.51 depending on the matrix and tile size. This happens because the naive approach already enjoys near-perfect hardware prefetching along its linear access patterns, whereas tiling disrupts these sequential accesses with frequent tile boundary crossings, reducing prefetcher efficiency.
2. **L1-D MPKI Variation (Matrix and Tile sizes):** As matrix size increased, the MPKI for smaller tiles (like 16×16) spiked significantly (e.g., to 0.51 at 1024^2), indicating that the small tile overhead and boundary fetches dominated. Larger tiles (like 64×64 or 128×128) stabilized the MPKI around 0.11–0.18. This is because larger tiles better amortize the boundary overhead while still remaining small enough to fit their working sets within the L1 or L2 caches.
3. **Speedup and Limiting Factors:** Tiling alone achieved negligible to small speedups (ranging from $\sim 0.94\times$ to $1.78\times$). It was limited by the lack of Instruction-Level Parallelism (ILP) and the scalar nature of the inner loop, alongside the added arithmetic overhead of computing tile boundaries. To truly unlock performance, tiling must be paired with loop unrolling or SIMD vectorization.

5 Task 1D: SIMD with AVX2

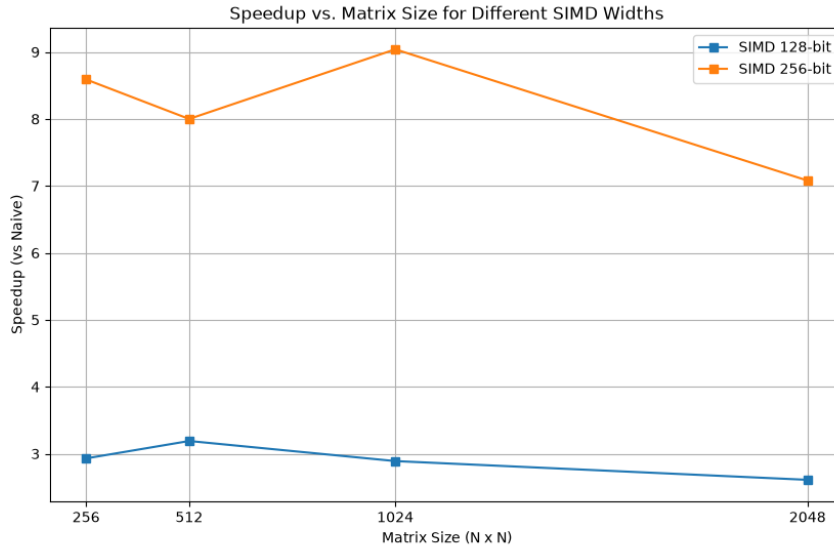


Figure 5: Speedup vs. Matrix Size for different SIMD widths.

Eight consecutive output columns are computed in a single 256-bit AVX2 vector operation using `_mm256_fmadd_ps`.

```
1 for (int ox = 0; ox < W; ox += 8) {
2     _mm256 res = _mm256_setzero_ps();
3     for (int ky = 0; ky < K; ++ky)
4         for (int kx = 0; kx < K; ++kx) {
5             _mm256 wt = _mm256_set1_ps(ker[ky*K+kx]);
6             _mm256 in_vec = _mm256_loadu_ps(&in[(oy+ky)*in_stride+(ox+kx)]);
7             res = _mm256_fmadd_ps(in_vec, wt, res);
8         }
9     _mm256_storeu_ps(&out[oy*W+ox], res);
10 }
```

5.1 Analysis

The SIMD implementation provides a massive performance boost because each AVX2 `vfmadd231ps` instruction processes 8 floats simultaneously. This gives an $8\times$ increase in compute density. The kernel weight is broadcast once per (ky, kx) pair and shared across all 8 output columns in the vector. Observed speedup is **7.96–10.05 $\times$** . The L1D MPKI drops to 0.08–0.17 because each 32-byte vector load fetches 8 spatially adjacent floats at once, making highly efficient use of memory and cache lines.

Deliverable Answers:

1. **Instruction Count Changes (Naive to SIMD):** The instruction count for the SIMD implementation remained relatively similar or slightly higher (~ 1.2 – 1.3 billion instructions at 1024^2) compared to naive (~ 1.1 billion). Even though each SIMD instruction does 4–8 $\times$ more work, the total instruction count did not drop proportionally because of the loop overhead, unrolling differences, and the setup instructions (like `_mm256_setzero_ps` and `_mm256_set1_ps`) required inside the loops.
2. **Speedup and Contributing Factors:** We achieved massive speedups: ~ 2.6 – $3.2\times$ with 128-bit SIMD, and ~ 7.1 – $9.0\times$ with 256-bit SIMD (AVX2). This is driven almost entirely by data-level parallelism—processing 4 or 8 single-precision floating-point numbers per cycle—and heavily utilizing FMA (Fused Multiply-Add) which does an addition and multiplication in a single hardware step.
3. **SIMD Intrinsics Used:** We used `_mm256_setzero_ps` to initialize accumulators, `_mm256_set1_ps` to broadcast the single kernel weight across the vector, `_mm256_loadu_ps` for unaligned loads of the input matrix, `_mm256_fmadd_ps` to efficiently multiply and accumulate, and `_mm256_storeu_ps` to write back the results. These were chosen to maximize data throughput and utilize the efficient FMA units on the processor.

6 Task 1E: Optimized — SIMD + Unrolling

The final kernel combines AVX2 with an 8-wide unroll in the column direction, processing 64 output columns per iteration using 8 independent `_mm256` accumulators.

```
1 for (; ox <= W - 64; ox += 64) {
2     _mm256 acc0=_mm256_setzero_ps(), acc1=_mm256_setzero_ps(),
3         acc2=_mm256_setzero_ps(), acc3=_mm256_setzero_ps(),
4         acc4=_mm256_setzero_ps(), acc5=_mm256_setzero_ps(),
5         acc6=_mm256_setzero_ps(), acc7=_mm256_setzero_ps();
6     for (int ky = 0; ky < K; ++ky)
7         for (int kx = 0; kx < K; ++kx) {
8             _mm256 w = _mm256_set1_ps(ker[ky*K+kx]);
9             acc0 = _mm256_fmadd_ps(_mm256_loadu_ps(&in[...+ox+kx   ]), w, acc0);
10            acc1 = _mm256_fmadd_ps(_mm256_loadu_ps(&in[...+ox+kx+ 8]), w, acc1);
11            // ... acc2 through acc7
12        }
13    _mm256_storeu_ps(&out[oy*W+ox   ], acc0); // ... store acc1--acc7
14 }
15 // scalar AVX tail for remaining columns
```

6.1 Analysis

This combined optimization leverages both SIMD parallelism and multiple independent accumulators. By unrolling 8-wide and using 8 independent AVX accumulators, FMA latency is fully hidden and both AVX FMA ports are kept saturated. This achieves **14–16.6 $\times$** speedup for moderate matrix sizes. At 2048^2 , the speedup drops to $9.5\times$ because the 8-wide unroll issues 8 simultaneous 32-byte loads from different column positions, heavily stressing memory bandwidth. The L1D MPKI rises to 0.50 at this size, confirming a shift to a memory-bound regime for the largest matrix.

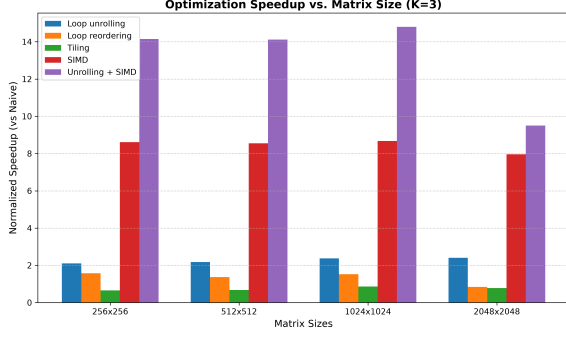


Figure 6: Speedup vs. Matrix Size for all optimization techniques ($K = 3$).

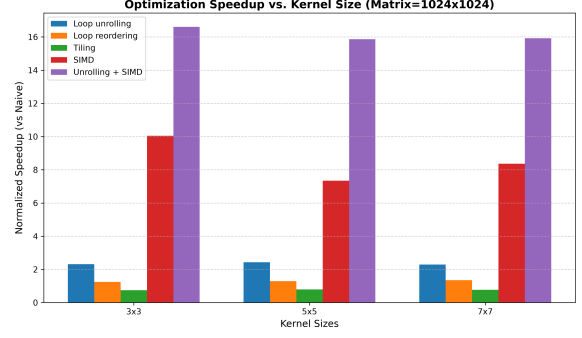


Figure 7: Speedup vs. Kernel Size (1024×1024).

7 Summary of Results

Variant	256^2	512^2	1024^2	2048^2
Loop Reordering	$1.58\times$	$1.37\times$	$1.53\times$	$0.84\times$
Loop Unrolling	$2.10\times$	$2.18\times$	$2.37\times$	$2.40\times$
Tiling	$0.66\times$	$0.68\times$	$0.86\times$	$0.78\times$
SIMD	$8.61\times$	$8.55\times$	$8.67\times$	$7.96\times$
Unrolling + SIMD	$14.15\times$	$14.12\times$	$14.80\times$	$9.50\times$

Table 1: Speedup over naive for varying matrix sizes, $K = 3$.

Variant	K=3	K=5	K=7
Loop Reordering	$1.25\times$	$1.30\times$	$1.35\times$
Loop Unrolling	$2.32\times$	$2.43\times$	$2.29\times$
Tiling	$0.75\times$	$0.80\times$	$0.77\times$
SIMD	$10.05\times$	$7.34\times$	$8.36\times$
Unrolling + SIMD	$16.61\times$	$15.87\times$	$15.92\times$

Table 2: Speedup over naive for varying kernel sizes, Matrix = 1024×1024 .

8 Key Takeaways

- **Loop Reordering** helps only for small matrices where the working set fits in cache. For large matrices it thrashes the cache, making it slower than naive. Its MPKI at 2048^2 (1.72) is the highest of all variants.
- **Loop Unrolling** provides a rock-steady $\sim 2.3\times$ speedup at all sizes. It hides FP-add latency via ILP without changing the memory footprint.
- **Tiling** alone is counterproductive without vectorization. It adds tile-boundary overhead and breaks the sequential access pattern exploited by the hardware prefetcher, resulting in $\text{sub-}1\times$ speedup.
- **SIMD (AVX2)** is the single largest lever, delivering $\sim 8\times$ speedup by processing 8 floats per instruction. L1D MPKI drops to 0.08–0.17, the lowest of all variants.
- **Unrolling + SIMD** is the best overall: 8 independent AVX accumulators hide FMA latency while SIMD maximizes compute width, achieving up to **$16.6\times$ speedup**. Performance degrades at 2048^2 as the kernel becomes memory-bandwidth limited.